Remaking 3DGS

Elijah Clark, Nick Welsh

Problem

3DGS, 3D Gaussian Splatting, is a method of 3d reconstruction using 3D gaussians. The goal of 3DGS is to predict the in-between image between input images and poses.

The main implementation of 3DGS is implemented in python and CUDA, so it requires a lot of setup and is not cross platform. As a result, it is slow to train and not easy to modify.

In this project, we remade 3DGS using Vulkan and C++ to solve these problems, and to learn about implementing AI through low-level programming languages.

The submitted version of the code is our first iteration on this problem, and there are several ways to optimize our solution.

Our goal for this submission is to derive the loss gradient for 3DGS and to implement all framework through low level libraries.

Dataset

We used https://www.tanksandtemples.org as our dataset. We trained on a subset of 'Family' image set.

We first put the dataset through COLMAPS. Then, we downsized each image by $\frac{1}{6}$, resulting in a resolution of 320x180. Finally, we put the images and generated point cloud through our 3DGS model.

Code

Our code is in github.com/stamparkour/gsplat.

The output images are in github.com/stamparkour/gsplat/tree/main/testset/out.

The training logs are in github.com/stamparkour/gsplat/blob/main/testset/out_log.txt.

Architecture

The parameters that are trained:

- quaternion/rotation of the gaussian
- scale of the gaussian
- mean/position of the gaussian

These parameters are converted to the inverse covariance matrix.

$$\Sigma^{-1} = RS^{-2}R^{\top}$$

The 3DGS forward pass is:

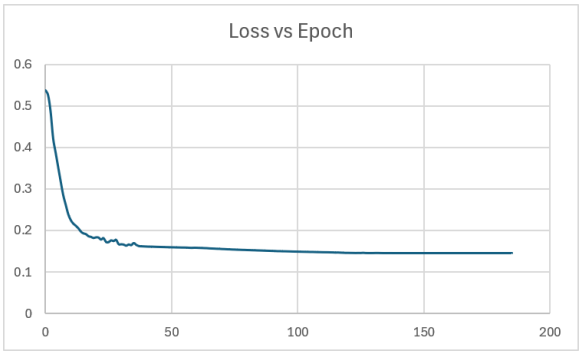
1. Project 3D gaussians into screen space. $\Sigma'^{-1} = (J^{-1})^{\top}(W^{-1})^{\top}\Sigma^{-1}W^{-1}J^{-1}$
2. Sort the gaussians based off depth.
3. Render all gaussians based the order and alpha values.
$$\alpha = e^{-\frac{1}{2}x^{\top}\Sigma'^{-1}x}$$
4. Add the background color

Loss is then calculated by the squared difference between the actual pixel color and the predicted color.

The backwards pass modifies the parameters of each gaussian using the gradient of the loss function. For each parameter, we sum the gradient of each pixel through the covariance matrix and the parameters characteristic matrix.

We batch several images into one gradient pass to better utilize parallel computing.

Experimental Evaluation



Graph 1 loss over time.



Image 1 Target image



Images 2 output at epoch 1, 5, 50, 185.

Our implementation of 3DGS does decrease the loss over several iterations, but the model plateaus very early on. Our model restricts the number of gaussians to the initial count to simplify the design. As a result, we are unable to get realistic 3D reconstructions.

For our dataset, the output stabilized at around epoch 150 with no significant gains in loss. Qualitatively, the output images look like blurry versions of their source images, so the model is correctly adjusting the gaussians to match the target, but the parameter space is too small to fully reconstruct the scene.

Notably, several gaussians in the left half of the image do not change much throughout training, resulting in a patchy set of brighter dots. We believe that this is due to the size difference of the gaussians, resulting in the magnitude of the gradient being orders of magnitude apart. This will require more testing to verify this claim.

Further, our implementation uses one color per gaussian, so directional details will not show.

The hyperparameters we used are:

- 3930 gaussians
- gaussian scale initialized with 0.02
- gaussian alpha initialized with 0.1
- 16 images of 320x180 resolution
- learning rate of 1.0
- image batch size of 5

Computational Performance

The model trained 185 epochs in around 20 minutes on an RTX 3060 TI.

Breakdown of CPU time:

- 35% waiting for GPU after invoking shader
- ~30% managing memory
- ~30% invoking Vulkan API
- <5% CPU-side gaussian computation

These results show that the CPU should be better utilized. This implementation only uses one CPU thread, so there is a lot of extra computation that the CPU can take up.

Process Evaluation

We had no major challenges with implementing 3DGS except for the limited time. As a result, we had to simplify our implementation of 3DGS.

We have met our goals for this iteration of the project:

- implement a simplified version of gaussian splatting
- learn how to use Vulkan and Slangc
- output images to qualitatively verify our implementation

We believe that with a few more iterations, we could make a completive 3DGS implementation that allows for experimentation and further expansion by other people.

Future changes:

- Reimplement Vulkan to better utilize parallel computing, use multiple GPUs, and minimize API calls.
- Reimplement pipeline to be more modular and easy to make changes.
- Create a 3d gaussian splatting viewer.
- Implement removing and adding gaussians dynamically.
- Implement nonlinear alpha.
- Implement further color information.
- Implement python bindings to allow for quick prototyping.

All in all, we are going to have to reimplement most of the code to be better suited for high throughput applications.

Insights

In conclusion, we implemented 3DGS to learn about the algorithms behind it and to implement ai systems using low-level libraries. We have met both goals.

Notably, Eli learnt:

- Slang shading language
- Vulkan API
- derivation of 3DGS gradient
- gaussian splatting bottlenecks